

No **JavaScript**, o prototype é um mecanismo fundamental que permite **herança e compartilhamento de métodos e propriedades** entre objetos. Ele é parte central do modelo de objetos do JavaScript, que é baseado em **prototipagem**, diferente de linguagens como Java ou C#, que usam **classes** (embora hoje JavaScript também tenha `class`, por baixo dos panos ainda usa prototype).

Vamos por partes:

1. O que é o prototype

Quando você cria uma **função construtora** em JavaScript, automaticamente ela ganha uma propriedade chamada prototype.

Essa propriedade é um **objeto** que contém métodos e propriedades que serão **compartilhados** por todas as instâncias criadas a partir dessa função.

Exemplo simples:

```
function Pessoa(nome, idade) {
  this.nome = nome;
  this.idade = idade;
}

// Adicionando um método ao prototype
Pessoa.prototype.apresentar = function() {
  console.log(`Olá, meu nome é ${this.nome} e tenho
  ${this.idade} anos.`);
};

const p1 = new Pessoa("Ana", 25);
const p2 = new Pessoa("João", 30);

p1.apresentar(); // Olá, meu nome é Ana e tenho 25 anos.
p2.apresentar(); // Olá, meu nome é João e tenho 30 anos.
```

Por que isso é bom?

O método apresentar **não será duplicado** em cada instância.

Ele fica no prototype, e as instâncias apenas **herdam a referência** a esse método, economizando memória.

2. A cadeia de protótipos (Prototype Chain)

Quando você acessa uma propriedade ou método de um objeto, o JavaScript **procura primeiro no próprio objeto**. Se não encontrar, ele **sobe a cadeia de protótipos** (prototype chain), buscando no protótipo, depois no protótipo do protótipo, e assim por diante, até chegar em `Object.prototype`.

```
console.log(p1.toString);  
// Função nativa herdada de Object.prototype
```

A ordem de busca seria:

```
p1 → Pessoa.prototype → Object.prototype → null
```

3. Visualizando o prototype

Cada objeto em JavaScript tem uma **referência interna** chamada `[[Prototype]]`, que pode ser acessada com:

- `Object.getPrototypeOf(obj)` → forma recomendada
- `__proto__` → forma antiga (ainda existe, mas não é ideal usar)

Exemplo:

```
console.log(Object.getPrototypeOf(p1) === Pessoa.prototype); // true
```

Ou diretamente:

```
console.log(p1.__proto__ === Pessoa.prototype); // true
```

4. Prototype vs class

Desde o ES6, temos a sintaxe de `class` no JavaScript. Porém, **por baixo dos panos**, continua sendo prototype.

```
class Pessoa {
```

```

    constructor(nome, idade) {
        this.nome = nome;
        this.idade = idade;
    }

    apresentar() {
        console.log(`Olá, meu nome é ${this.nome} e tenho ${this.idade}
anos.`);
    }
}

const p1 = new Pessoa("Maria", 40);
console.log(Object.getPrototypeOf(p1)); // Pessoa.prototype

```

O código acima é **equivalente** ao exemplo anterior usando função construtora + prototype.

5. Adicionando métodos depois da criação

Você pode adicionar métodos ao prototype **mesmo depois de ter instâncias criadas**:

```

Pessoa.prototype.saudacao = function() {
    console.log("Olá!");
};

p1.saudacao(); // Olá!

```

Isso funciona porque todas as instâncias estão ligadas ao mesmo prototype.

6. Sobrescrevendo propriedades do prototype

Se uma instância tiver uma **propriedade própria** com o mesmo nome que existe no prototype, a do próprio objeto será usada, sem alterar o prototype.

```

p1.saudacao = function() {
    console.log("Oi, eu sou especial!");
};

p1.saudacao(); // Oi, eu sou especial!

```

O método original no prototype ainda existe, mas fica "oculto" para essa instância específica.

7. Quando usar prototype

- Quando você quer **compartilhar métodos entre muitas instâncias** sem duplicar código.
- Para implementar **herança** em JavaScript.
- Ao criar **bibliotecas ou frameworks**, onde a performance e uso de memória são importantes.

Vamos criar um exemplo prático de **herança usando prototype** no JavaScript para entender bem como funciona a cadeia de protótipos.

1. Cenário do exemplo

Vamos ter duas "classes":

- **Pessoa** → classe base (pai)
 - Propriedades: nome e idade
 - Método: `apresentar()`
- **Aluno** → classe derivada (filha)
 - Propriedades herdadas de Pessoa
 - Propriedade extra: `curso`
 - Método novo: `estudar()`

Queremos que **Aluno herde tudo de Pessoa**, sem duplicar código.

2. Criando a classe base (Pessoa)

```
function Pessoa(nome, idade) {  
  this.nome = nome;  
  this.idade = idade;  
}
```

```
}

Pessoa.prototype.apresentar = function() {
  console.log(`Olá, meu nome é ${this.nome} e tenho ${this.idade}
anos.`);
};
```

3. Criando a classe derivada (Aluno)

Agora vamos criar Aluno, que precisa **herdar de Pessoa**.

```
function Aluno(nome, idade, curso) {
  // Chama o construtor de Pessoa para inicializar nome e idade
  Pessoa.call(this, nome, idade);
  this.curso = curso;
}
```

Pessoa.call(this, nome, idade)

- call executa a função Pessoa **no contexto do this de Aluno**,
- Assim, nome e idade ficam como propriedades diretas do objeto Aluno.

4. Ligando o prototype do Aluno ao de Pessoa

Agora precisamos dizer que **Aluno herda os métodos de Pessoa**.

```
Aluno.prototype = Object.create(Pessoa.prototype);
Aluno.prototype.constructor = Aluno;
```

O que acontece aqui:

1. Object.create(Pessoa.prototype)
 - Cria um **novo objeto vazio** cujo prototype é Pessoa.prototype.

- Isso garante que todos os métodos de Pessoa sejam herdados.

2. `Aluno.prototype.constructor = Aluno`

- Ajusta a propriedade `constructor`, que por padrão ficaria apontando para `Pessoa`.

5. Adicionando método específico de Aluno

Agora vamos dar ao Aluno um método próprio.

```
Aluno.prototype.estudar = function() {  
  console.log(`${this.nome} está estudando ${this.curso}.`);  
};
```

6. Testando a herança

```
const aluno1 = new Aluno("Maria", 20, "Engenharia");  
  
aluno1.apresentar(); // Olá, meu nome é Maria e tenho 20 anos.  
aluno1.estudar();    // Maria está estudando Engenharia.
```

O `aluno1` herdou o método `apresentar` de `Pessoa` **via prototype**, e também tem o método `estudar` próprio de `Aluno`.

7. Visualizando a cadeia de protótipos

Podemos inspecionar a hierarquia:

```
console.log(Object.getPrototypeOf(aluno1) === Aluno.prototype); // true  
console.log(Object.getPrototypeOf(Aluno.prototype) === Pessoa.prototype);  
// true  
console.log(Object.getPrototypeOf(Pessoa.prototype) ===  
Object.prototype); // true
```

A cadeia fica assim:

`aluno1`

```
↓  
Aluno.prototype  
↓  
Pessoa.prototype  
↓  
Object.prototype  
↓  
Null
```

8. Exemplo completo

```
function Pessoa(nome, idade) {  
  this.nome = nome;  
  this.idade = idade;  
}  
  
Pessoa.prototype.apresentar = function() {  
  console.log(`Olá, meu nome é ${this.nome} e tenho ${this.idade}  
anos.`);  
};  
  
function Aluno(nome, idade, curso) {  
  Pessoa.call(this, nome, idade); // herda nome e idade  
  this.curso = curso;  
}  
  
Aluno.prototype = Object.create(Pessoa.prototype);  
Aluno.prototype.constructor = Aluno;  
  
Aluno.prototype.estudar = function() {  
  console.log(`${this.nome} está estudando ${this.curso}.`);  
};  
  
// Testando  
const aluno1 = new Aluno("Maria", 20, "Engenharia");  
aluno1.apresentar(); // Olá, meu nome é Maria e tenho 20 anos.  
aluno1.estudar();    // Maria está estudando Engenharia.
```

9. Comparando com class

A mesma coisa usando class fica bem mais simples, mas funciona **exatamente igual por baixo dos panos**:

```
class Pessoa {
  constructor(nome, idade) {
    this.nome = nome;
    this.idade = idade;
  }

  apresentar() {
    console.log(`Olá, meu nome é ${this.nome} e tenho ${this.idade}
anos.`);
  }
}

class Aluno extends Pessoa {
  constructor(nome, idade, curso) {
    super(nome, idade); // chama o construtor da classe pai
    this.curso = curso;
  }

  estudar() {
    console.log(`${this.nome} está estudando ${this.curso}.`);
  }
}

const aluno2 = new Aluno("João", 22, "Direito");
aluno2.apresentar(); // Olá, meu nome é João e tenho 22 anos.
aluno2.estudar();    // João está estudando Direito.
```

Resumo

Conceito	Prototype (Antigo)	Class (Novo)
Criação do pai	function Pessoa(...)	class Pessoa {...}
Ligação de herança	Aluno.prototype = Object.create(Pessoa.prototype)	class Aluno extends Pessoa
Chamar construtor pai	Pessoa.call(this, ...)	super(...)

Método compartilhado	Pessoa.prototype.metodo = function() {}	Dentro do corpo da classe
----------------------	--	---------------------------